General Information

Practical Exam: SWD392 –SU26

Duration: 85 Minutes

Instructions

- Internet access is **NOT allowed during the exam**.
- You may use **UML modeling tools in offline mode** (e.g., Visual Paradigm, Visio, Draw.io).

Business Overview

MediCare Hub is an online platform for managing healthcare appointments, supporting patients, doctors, and medical staff in scheduling and managing medical consultations.

Patients can look up available doctors, review doctor schedules, and book medical appointments for a specific date and time. Doctors manage their consultation schedules and record appointment outcomes after each visit. Receptionists handle appointment confirmations, check patient attendance, and process payments.

Every appointment carries a status throughout its lifecycle such as Pending, Confirmed, CheckedIn, Completed, or Cancelled. The system automatically updates doctor availability and appointment status according to the healthcare workflow.

This exam is designed to assess your ability to design software architecture and UML models rather than to implement program logic.

The examination evaluates students' understanding of software architecture and UML-based system design. Students are not required to write any source code, and all answers must be presented using UML diagrams.

Students may choose either **Layered Architecture** or **Microservices Architecture**, but the chosen architecture must be applied consistently across all diagrams. **The Repository Pattern is mandatory.**

Students may make reasonable assumptions where needed, as long as those assumptions do not contradict the given business context.

Question 1 (4.0 points) – Class Diagram

You need to design a design-level **Class Diagram** for the Smart Healthcare Appointment System.

The system comprises multiple user roles whose responsibilities are partly shared and partly specialized. Patients book appointments with doctors, while receptionists handle appointment confirmations and payment operations.

Each appointment is associated with exactly one patient and one doctor. Payments may be required for medical consultations. Doctors also maintain consultation schedules and appointment records.

To keep the architecture clean and ensure separation of concerns, business logic is handled by service classes, while data access operations are handled by repository classes.

Your Class Diagram must satisfy the following requirements:

- `User` is a base class.
- `Patient`, `Doctor`, and `Receptionist` are specialized types of `User` using inheritance.
- An `Appointment` is created by a patient.
- A doctor can manage multiple appointments.
- Each `Appointment` has a date, time slot, and status.
- Each `Appointment` is associated with exactly one `Payment`.
- A `DoctorSchedule` stores doctor availability information.
- Business logic related to appointment booking, payment, and schedule management is handled by service classes.
- Repository classes must implement a generic `Repository<T>` interface.
- The service layer depends on repository classes for data access.

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

UML relationships that must be clearly shown:

- Inheritance and interface implementation
- Association, aggregation, and composition where appropriate
- Dependency relationships between service and repository layers
- Clear attributes, methods, and multiplicities

Question 2 (4.0 points) – Sequence Diagram

Use Case: Book Appointment and Pay Consultation Fee

This question focuses on the interaction flow that takes place when a patient books a medical appointment and pays the consultation fee.

The process starts when the patient selects a doctor and requests an appointment for a specific time slot. The system then checks whether the doctor is available during that period.

If the doctor is available, an appointment is created and the patient moves on to payment. If the payment succeeds, the appointment is confirmed; otherwise, the appointment is cancelled.

Your Sequence Diagram must meet the following requirements:

- The diagram must follow either **Layered Architecture** or **Microservices Architecture** consistently.
- The interaction flow must clearly include:
  - A user-facing component (UI / Patient Portal)
  - A controller or service component handling business logic
  - Repository components responsible for data access
  - Domain entities such as `Appointment`, `Doctor`, and `Payment`
- Repository Pattern usage must be explicit through repository method calls.
- Conditional behavior must be modeled using `alt` fragments for:
  - Doctor availability
  - Payment success or failure

Question 3 (3.0 points) – State Diagram

Target: Appointment Lifecycle Management

Each appointment in the healthcare system passes through multiple states from creation to completion or cancellation. The system must track appointment status to ensure proper coordination between patients, doctors, and receptionists.

You need to design a **State Diagram** for the `Appointment` entity, describing the complete lifecycle of an appointment.

Your State Diagram must satisfy the following requirements:

- The diagram must include a clearly defined initial state and final state.
- The diagram must include at least the following states (state names may vary, but meaning must be preserved):
  - `Pending` – the appointment has been created but not yet confirmed
  - `Confirmed` – the appointment has been confirmed successfully
  - `CheckedIn` – the patient has arrived at the clinic
  - `InConsultation` – the doctor is examining the patient
  - `Completed` – the appointment has finished successfully
- The diagram must include an alternative termination state such as:
  - `Cancelled`
  - `PaymentFailed`
- At least six state transitions must be shown, each with a clear triggering event, such as:
  - `createAppointment`
  - `paySuccess`
  - `payFail`
  - `confirmAppointment`
  - `checkIn`
  - `startConsultation`
  - `finishConsultation`
  - `cancelAppointment`
- At least one branching path must represent payment failure or appointment cancellation.

END OF EXAMINATION

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*